PROJECT DEVELOPER & VIVA GUIDE

Cyber Threat Detection and Incident Response Dashboard

A codebase-grounded academic guide for development, demonstration, architecture explanation, and viva preparation.

`Repository analysed: cyber-threat-dashboard-full/cyber-threat-dashboard`

Scope: FastAPI • React TypeScript • MongoDB • Elasticsearch • Docker Compose • rule-based detection • Isolation Forest anomaly detection

Prepared from the implementation files in this repository. It distinguishes implemented behavior from future extension points.

1. Table of Contents

- 1. Table of Contents
- 2. Part A — Module-by-module developer guide
- 3. Part B — Complete end-to-end workflow
- 4. Part C — Feature deep-dives
- 5. Part D — Viva learning guide
- 6. Appendix — API map, data stores, and demo checklist

Reading note: file paths and function/API names are styled in monospaced type throughout. MongoDB collection and Elasticsearch index names are deliberately identified separately because they serve different responsibilities.

2. Part A — Module-by-module developer guide

Every module below answers the ten requested developer questions using the actual project structure and implementation behavior.

2.1 Application entry point

`backend/app/main.py`

Question	Project-specific answer
1. What is this module?	The FastAPI composition root. It creates the application, configures CORS and global error handling, owns lifecycle startup/shutdown, and mounts every version-1 router.
2. Why does it exist?	It gives the backend one controlled boot sequence. Mongo indexes, Elasticsearch templates, persisted rule states, and the ingestion-to-detection callback must be ready before requests arrive.
3. What problem does it solve?	Without a composition root, routers and infrastructure would initialize independently and the upload pipeline could ingest logs without automatic detection.
4. How does it interact with other modules?	On startup it calls Mongo connect/index setup, attempts Elasticsearch connect, registers *handle_ingested_logs* on *default_pipeline*, loads Mongo rule configuration, then mounts auth, logs, incidents, analytics, rules, detections, and ML routers.
5. What happens if removed?	The API would have no ASGI application, no routes, no startup connections, and no automatic pipeline callback. The React client could not use the service.
6. Which files are responsible?	Primary responsibility is in `backend/app/main.py`.
7. Which APIs use it?	/health, /health/ready, and all /api/v1/* routes through router inclusion.
8. Which Mongo collections / ES indices?	Directly checks MongoDB and Elasticsearch on */health/ready*; does not own a collection or index.
9. What data enters?	Environment-derived settings, HTTP requests, and FastAPI lifespan events.
10. What data leaves?	Mounted REST API, standardized 500 response, liveness/readiness JSON, and initialized process-wide services.

2.2 Core configuration, security, and dependencies

`backend/app/core/config.py; backend/app/core/security.py; backend/app/core/dependencies.py; backend/app/core/logging_config.py`

Question	Project-specific answer
1. What is this module?	Cross-cutting platform layer: typed environment settings, bcrypt password functions, JWT issue/validation, OAuth bearer extraction, and role-based dependency checks.
2. Why does it exist?	Security rules should be centrally auditable instead of duplicated inside each endpoint.
3. What problem does it solve?	It solves secret/config sprawl, inconsistent token validation, and endpoints accidentally trusting unauthenticated callers.
4. How does it interact with other modules?	Auth router uses hashing and token creation; protected routers use *get_current_user* or *require_role*; database clients read connection settings; main reads CORS and app metadata.
5. What happens if removed?	Login cannot safely issue tokens, RBAC cannot protect upload/admin functions, and modules would need unsafe direct environment access.
6. Which files are responsible?	Primary responsibility is in `backend/app/core/config.py; backend/app/core/security.py; backend/app/core/dependencies.py; backend/app/core/logging_config.py`.
7. Which APIs use it?	Indirectly protects all authenticated APIs; admin-only examples include *POST /ml/train*, *PATCH /rules/{rule_name}*, and *POST /auth/register*.
8. Which Mongo collections / ES indices?	Reads Mongo *users* while resolving a token; no Elasticsearch index.

Question	Project-specific answer
9. What data enters?	.env variables, Bearer tokens, plaintext login password, and requested allowed roles.
10. What data leaves?	Settings object, password hashes/verification result, signed access/refresh JWTs, current *UserInDB*, or HTTP 401/403.

2.3 API v1 HTTP layer

`backend/app/api/v1/auth.py; logs.py; detections.py; incidents.py; analytics.py; rules.py; ml.py`

Question	Project-specific answer
1. What is this module?	Thin FastAPI controllers grouped by bounded context under the */api/v1* prefix.
2. Why does it exist?	It separates HTTP validation/authorization/response shaping from parsing, persistence, detection, and incident business logic.
3. What problem does it solve?	It provides a stable contract for the frontend and external clients while preventing UI code from reaching data stores directly.
4. How does it interact with other modules?	Routers depend on core authentication, invoke ingestion/detection/incident services, query repositories, and return Pydantic/JSON response shapes consumed by frontend services.
5. What happens if removed?	The business classes might exist, but neither the React UI nor a shipper can call them over HTTP.
6. Which files are responsible?	Primary responsibility is in `backend/app/api/v1/auth.py; logs.py; detections.py; incidents.py; analytics.py; rules.py; ml.py`.
7. Which APIs use it?	Auth: login/refresh/register/me; Logs: upload/ingest/search/batches; Detections: list/alerts/rules/run; ML: train/predict/models/evaluate; Incidents: list/detail/status/notes/assign; Analytics: summary/top-ips/severity/timeline/attack-types/recent-events; Rules: list/update.
8. Which Mongo collections / ES indices?	Through services: all Mongo collections and the *security-logs-** Elasticsearch indices.
9. What data enters?	JSON bodies, multipart files, query parameters, and JWT-authenticated user context.
10. What data leaves?	HTTP status codes and frontend-ready JSON such as *{detections, count}*, *{incidents, count}*, upload counts, and analytics charts data.

2.4 Domain models and schemas

`backend/app/models/log_schema.py; detection_schema.py; alert_schema.py; incident_schema.py; user_schema.py`

Question	Project-specific answer
1. What is this module?	Pydantic contracts for normalized logs, upload batches, detection events, alerts, incidents, users, tokens, roles, and request bodies.
2. Why does it exist?	A shared typed language prevents parser, detector, database, and API layers from disagreeing about field names and legal values.
3. What problem does it solve?	It solves schema drift and validates required structure before data reaches downstream logic.
4. How does it interact with other modules?	Parsers produce *ParsedEvent*; normalizer creates *NormalizedLog*; rules/ML produce *DetectionEvent*; alerting and correlator use those events; API routes expose model-shaped data.
5. What happens if removed?	Validation and canonical contracts disappear; detection and storage would need ad-hoc dictionaries and failures would appear later and less clearly.
6. Which files are responsible?	Primary responsibility is in `backend/app/models/log_schema.py; detection_schema.py; alert_schema.py; incident_schema.py; user_schema.py`.
7. Which APIs use it?	All routes use these directly or indirectly; upload response is *LogUploadResponse*, login response is *TokenResponse*.
8. Which Mongo collections / ES indices?	Schema maps to Mongo users/incidents/detections/alerts/ingestion_batches and ES normalized log documents.

Question	Project-specific answer
9. What data enters?	Raw parser results, request models, database documents, model scores, and incident workflow input.
10. What data leaves?	Validated, serialized objects with enum values such as source type, severity, engine, role, and incident status.

2.5 MongoDB repository

`backend/app/db/mongo.py`

Question	Project-specific answer
1. What is this module?	Async Motor connection owner and repository functions for operational, identity, workflow, detection, alert, rule, and audit metadata.
2. Why does it exist?	MongoDB suits mutable document-style operational records and aggregates; the repository keeps Motor out of business and HTTP code.
3. What problem does it solve?	It provides persistent users, incidents, detection results, alerts, rule settings, and ingestion audit records.
4. How does it interact with other modules?	Core dependencies retrieve users; auth creates users/updates logins; pipeline saves batches; engine saves detections/alerts; correlator and incident service update incidents; analytics aggregates Mongo results.
5. What happens if removed?	Authentication user lookup, incident history, alert/detection persistence, configurable rule state, audit batches, and dashboard counts fail.
6. Which files are responsible?	Primary responsibility is in `backend/app/db/mongo.py`.
7. Which APIs use it?	Auth, logs/batches, detections/alerts, incidents, analytics, and rules APIs use it indirectly.
8. Which Mongo collections / ES indices?	Collections: *users*, *incidents*, *detection_rules*, *system_config*, *ingestion_batches*, *detections*, *alerts*, and *counters*.
9. What data enters?	User credentials after hashing, incident documents, detection/alert Pydantic dumps, rule updates, upload batch metadata, and aggregation filters.
10. What data leaves?	Typed users, documents/lists/counts/aggregates, monotonic incident sequence numbers, and write acknowledgements.

2.6 Elasticsearch log repository

`backend/app/db/elastic.py; elasticsearch/mappings/security_logs_mapping.json`

Question	Project-specific answer
1. What is this module?	Async Elasticsearch client owner for time-oriented normalized log indexing, search, deduplication, and log-derived aggregation.
2. Why does it exist?	Logs are searched and aggregated by time/IP/text more often than they are updated; daily ES indices fit that workload better than Mongo operational collections.
3. What problem does it solve?	It provides scalable evidence search, ingest deduplication, live monitoring, top-IP and severity views.
4. How does it interact with other modules?	Pipeline uses cross-index duplicate checks and bulk indexing; logs API searches it; detections manual/ML training retrieve a time window; analytics runs aggregations.
5. What happens if removed?	Raw evidence cannot be stored/searchable, ingestion cannot complete, and analytics/features depending on telemetry fail.
6. Which files are responsible?	Primary responsibility is in `backend/app/db/elastic.py; elasticsearch/mappings/security_logs_mapping.json`.
7. Which APIs use it?	GET /logs/search; POST /detections/run; POST /ml/train; POST /ml/predict; GET analytics summary/top-ips/severity endpoints.
8. Which Mongo collections / ES indices?	Daily *security-logs-YYYY.MM.DD* indices, queried with *security-logs-**; template is named from *security-logs* prefix.
9. What data enters?	NormalizedLog documents, search filters, time windows, source IP/source type/severity terms, and log-id lists.
10. What data leaves?	Indexed IDs, accepted/rejected bulk counts, search *{total, results}*, dedupe set, and aggregate counts/buckets.

2.7 Ingestion parser and normalization layer

`backend/app/ingestion/parsers/base_parser.py; linux_parser.py; windows_parser.py; firewall_parser.py; web_parser.py; backend/app/ingestion/normalizer.py`

Question	Project-specific answer
1. What is this module?	Source adapters convert four raw formats into a light ParsedEvent; normalizer then produces one canonical NormalizedLog and deterministic SHA-256 log ID.
2. Why does it exist?	Linux syslog, Windows JSON, firewall connection lines, and Apache/Nginx access logs are structurally incompatible, but detectors need one stable schema.
3. What problem does it solve?	It separates format extraction from canonical mapping and makes re-upload idempotent through content-based log IDs.
4. How does it interact with other modules?	Pipeline selects a parser from PARSER_REGISTRY, sends ParsedEvent to normalize, and passes NormalizedLog to ES/detection/ML.
5. What happens if removed?	The system has no trustworthy bridge from source-specific bytes to detection-ready fields; every detector would need four format implementations.
6. Which files are responsible?	Primary responsibility is in `backend/app/ingestion/parsers/base_parser.py; linux_parser.py; windows_parser.py; firewall_parser.py; web_parser.py; backend/app/ingestion/normalizer.py`.
7. Which APIs use it?	POST /logs/upload and POST /logs/ingest use it through the pipeline.
8. Which Mongo collections / ES indices?	No direct store; its NormalizedLog output is written to ES and its batch outcome later goes to Mongo.
9. What data enters?	UTF-8 raw content and source type/host. Windows accepts JSON array or NDJSON; Linux recognizes sshd/sudo; firewall recognizes ALLOW/DENY lines; web recognizes Combined Log Format.
10. What data leaves?	ParsedEvent list, normalized logs containing event fields/raw evidence/timestamps, and low-default or parser-suggested severity.

2.8 Ingestion orchestration pipeline

`backend/app/ingestion/pipeline.py`

Question	Project-specific answer
1. What is this module?	The single end-to-end ingestion coordinator implementing parse → normalize → validate → deduplicate → ES bulk index → Mongo batch audit → post-ingestion callback.
2. Why does it exist?	This sequence must be consistent for file upload and programmatic ingestion and must not be duplicated in controllers.
3. What problem does it solve?	It isolates bad records, prevents duplicate logs across daily indices, records traceability, and decouples detection using an injected callback.
4. How does it interact with other modules?	Logs API calls *default_pipeline.process*; it uses parser registry/normalizer/ES/Mongo; main injects *handle_ingested_logs* after startup.
5. What happens if removed?	Uploads would need bespoke orchestration, have inconsistent dedupe/audit behavior, and would not automatically invoke detection.
6. Which files are responsible?	Primary responsibility is in `backend/app/ingestion/pipeline.py`.
7. Which APIs use it?	POST /logs/upload and POST /logs/ingest.
8. Which Mongo collections / ES indices?	Reads ES *security-logs-** for duplicate IDs, writes fresh logs to today's ES index, writes batch metadata to Mongo *ingestion_batches*.
9. What data enters?	Raw content, source type, uploader username, optional source host and file name.
10. What data leaves?	LogUploadResponse with accepted/rejected/duplicates/errors/log_ids/batch_id and an asynchronous list of fresh logs for detection.

2.9 Rule detection engine

`backend/app/detection/engine.py; backend/app/detection/rules/base_rule.py`

Question	Project-specific answer
1. What is this module?	A configurable RuleEngine registry that runs enabled BaseRule strategies and combines their DetectionEvent output.
2. Why does it exist?	Known attack signatures and behavioral thresholds need transparent, auditable detection independent of ML availability.
3. What problem does it solve?	It detects repeat failed logins, port scans, web payload signatures, privilege escalation, and off-hours privileged logins with MITRE context.
4. How does it interact with other modules?	Main registers its post-ingest handler. It invokes rule classes, then also calls ML prediction, saves combined detections, generates alerts, and asks the incident correlator to group them.
5. What happens if removed?	Ingestion would still store logs but known attack patterns would not become detections, alerts, incidents, or dashboard threat counts.
6. Which files are responsible?	Primary responsibility is in `backend/app/detection/engine.py; backend/app/detection/rules/base_rule.py`.
7. Which APIs use it?	Automatic on upload; POST /detections/run manually scans a recent ES window; GET /detections/rules exposes live rule metadata.
8. Which Mongo collections / ES indices?	Writes Mongo *detections* through the handler; source evidence comes from ES normalized logs.
9. What data enters?	A list of NormalizedLog objects and persisted enabled/config state from Mongo.
10. What data leaves?	Rule DetectionEvent objects with rule name, confidence, severity, MITRE technique, entity, evidence IDs, and explanation.

2.10 Detection rules

`backend/app/detection/rules/ssh_brute_force.py; windows_failed_login.py; port_scan.py; sql_injection.py; xss.py; directory_traversal.py; privilege_escalation.py; suspicious_admin_login.py`

Question	Project-specific answer
1. What is this module?	Eight focused detection strategies, each carrying its own rule name, MITRE mapping, thresholds/config, evidence selection, explanation, and severity logic.
2. Why does it exist?	Attack types have materially different semantics: a port scan is a many-port network pattern while SQLi/XSS/traversal are per-request payload signatures.
3. What problem does it solve?	They transform normalized telemetry into actionable, explainable security findings rather than opaque raw logs.
4. How does it interact with other modules?	Engine instantiates these rules; rules use BaseRule grouping/window/pattern helpers; their DetectionEvent output flows to Mongo, alerts, incidents, and frontend.
5. What happens if removed?	The engine remains but has no substantive signatures/threshold logic and produces no rule-based detections.
6. Which files are responsible?	Primary responsibility is in `backend/app/detection/rules/ssh_brute_force.py; windows_failed_login.py; port_scan.py; sql_injection.py; xss.py; directory_traversal.py; privilege_escalation.py; suspicious_admin_login.py`.
7. Which APIs use it?	GET /rules, PATCH /rules/{rule_name}, GET /detections/rules, POST /detections/run, and automatic upload processing.
8. Which Mongo collections / ES indices?	Rule enabled/config state is Mongo *detection_rules*; results are Mongo *detections*, alerts can reach *alerts*, and evidence lives in ES.
9. What data enters?	NormalizedLog batches: Linux/Windows auth events, firewall denied connections, and decoded web request paths.
10. What data leaves?	DetectionEvent records. Key thresholds: Linux SSH ≥4 in 120s, Windows failed logins ≥3 in 120s, port scan ≥5 distinct ports in 60s.

2.11 ML anomaly detection

`backend/app/detection/ml/base_model.py; feature_extraction.py; isolation_forest_model.py; model_registry.py; prediction_service.py; training_service.py`

Question	Project-specific answer
1. What is this module?	An unsupervised Isolation Forest subsystem behind an AnomalyModel interface, with feature engineering, versioned joblib artifacts, training, prediction, and labeled evaluation.
2. Why does it exist?	Rules catch known patterns; ML adds a source-IP behavioral outlier signal for unusual activity that may not match a signature.
3. What problem does it solve?	It trains a repeatable model, prevents training on fewer than five distinct entities, produces explainable anomaly events, and persists versions rather than overwriting them.
4. How does it interact with other modules?	ML API loads historical ES logs; feature extraction builds eight features per source IP; training registers a model; prediction returns DetectionEvents to the same alert/correlation path as rules.
5. What happens if removed?	The dashboard loses AI anomaly counts and the system only detects explicitly coded patterns; core ingestion/rules still operate.
6. Which files are responsible?	Primary responsibility is in `backend/app/detection/ml/base_model.py; feature_extraction.py; isolation_forest_model.py; model_registry.py; prediction_service.py; training_service.py`.
7. Which APIs use it?	POST /ml/train, POST /ml/predict, GET /ml/models, GET /ml/models/latest, POST /ml/evaluate; prediction also runs automatically after ingest if a model exists.
8. Which Mongo collections / ES indices?	Reads ES logs; writes ML DetectionEvents/alerts/incidents to Mongo; artifacts and *registry_index.json* live in *ml/models* mounted by Docker.
9. What data enters?	Normalized logs grouped by source IP, hyperparameters (days/contamination), and labeled CSV for evaluation.
10. What data leaves?	Version metadata, precision/recall/F1 diagnostics, and ML DetectionEvents with raw score, 0–100 batch-relative risk, evidence IDs, and top-feature explanation.

2.12 Alerting

`backend/app/detection/alerting.py; backend/app/models/alert_schema.py`

Question	Project-specific answer
1. What is this module?	A focused in-application alert creator that converts high and critical DetectionEvents into persisted Alert documents.
2. Why does it exist?	Not every detection should interrupt an analyst; alert thresholding distinguishes stored signal from urgent signal.
3. What problem does it solve?	It provides a durable alert feed while keeping external email/Discord/Telegram delivery explicitly outside the implemented scope.
4. How does it interact with other modules?	Called by automatic engine handler and manual rule/ML routes after detections are saved; uses Mongo repository and audit logger.
5. What happens if removed?	Detections and incidents remain, but the dedicated alert list and critical alert dashboard count lose their source.
6. Which files are responsible?	Primary responsibility is in `backend/app/detection/alerting.py; backend/app/models/alert_schema.py`.
7. Which APIs use it?	GET /detections/alerts exposes persisted alerts; POST /detections/run and POST /ml/predict can create them.
8. Which Mongo collections / ES indices?	Writes Mongo *alerts*; AlertInDB includes *channel_dispatched* as a future integration seam.
9. What data enters?	DetectionEvent list.

Question	Project-specific answer
10. What data leaves?	One AlertInDB per high/critical event: title, threat type, severity, entity, detection ID, MITRE technique, description, timestamp.

2.13 Incident correlation and service

`backend/app/incidents/correlator.py; backend/app/incidents/service.py`

Question	Project-specific answer
1. What is this module?	Correlation groups related detection events into incidents; service enforces analyst workflow, notes, assignment, and non-destructive status transitions.
2. Why does it exist?	A burst of detections should not produce dozens of separate analyst tickets. Incidents are the SOC work unit, not raw logs.
3. What problem does it solve?	It deduplicates investigation workload by (entity value, threat key, active status), escalates severity, tracks evidence, and preserves audit history.
4. How does it interact with other modules?	Engine/manual runners invoke correlator after detections; service reads/writes Mongo; incidents router exposes the workflow to the UI.
5. What happens if removed?	Users would see isolated detections but could not manage a correlated case, assign it, record notes, or resolve it safely.
6. Which files are responsible?	Primary responsibility is in `backend/app/incidents/correlator.py; backend/app/incidents/service.py`.
7. Which APIs use it?	GET /incidents, GET /incidents/{ref}, PATCH status, POST notes, PATCH assign.
8. Which Mongo collections / ES indices?	Mongo *incidents* and *counters* for atomic *INC-1001*-style references; reads detection evidence already stored in Mongo/ES.
9. What data enters?	DetectionEvent lists, status/note/assignment request bodies, and authenticated actor username.
10. What data leaves?	Created/updated incident documents, workflow audit notes, severity escalation notes, and safe status transitions OPEN → INVESTIGATING → RESOLVED → CLOSED.

2.14 Frontend API and service layer

`frontend/src/api/client.ts; endpoints.ts; tokenStorage.ts; frontend/src/services/authService.ts; logService.ts; detectionService.ts; incidentService.ts; analyticsService.ts; ruleService.ts`

Question	Project-specific answer
1. What is this module?	Typed Axios boundary that centralizes API base URL, route strings, Bearer attachment, silent refresh/retry, browser token persistence, and service functions.
2. Why does it exist?	React components should describe UI state, not duplicate request URLs, headers, refresh logic, or response parsing.
3. What problem does it solve?	It solves cross-page consistency and prevents a 401 storm from causing multiple simultaneous refresh requests.
4. How does it interact with other modules?	Every feature page calls a service; Axios interceptor adds access JWT; a single in-flight refresh calls */auth/refresh*; tokenStorage chooses local or session storage for Remember Me.
5. What happens if removed?	Pages would need direct HTTP calls; auth refresh and endpoint naming would become fragmented and fragile.
6. Which files are responsible?	Primary responsibility is in `frontend/src/api/client.ts; endpoints.ts; tokenStorage.ts; frontend/src/services/authService.ts; logService.ts; detectionService.ts; incidentService.ts; analyticsService.ts; ruleService.ts`.
7. Which APIs use it?	Mirrors all backend v1 routes, especially /auth, /logs, /detections, /ml, /incidents, /analytics, and /rules.
8. Which Mongo collections / ES indices?	Browser localStorage/sessionStorage for access/refresh JWTs; no direct Mongo or ES access.
9. What data enters?	Form data, typed query filters, endpoint-specific request bodies, and tokens.
10. What data leaves?	Typed Promise response shapes for pages; authenticated HTTP requests; redirect to /login if refresh fails.

2.15 Frontend features and routes

`frontend/src/routes/AppRouter.tsx; frontend/src/features/auth/*; dashboard/pages/DashboardPage.tsx; logs/pages/UploadLogsPage.tsx; monitoring/pages/LiveMonitoringPage.tsx; detections/pages/ThreatDetectionPage.tsx; anomalies/pages/AnomalyDetectionPage.tsx; incidents/pages/IncidentManagementPage.tsx; analytics/pages/AnalyticsPage.tsx; rules/pages/DetectionRulesPage.tsx`

Question	Project-specific answer
1. What is this module?	React TypeScript presentation and route layer for authentication, dashboard, upload/search, detections, ML, incidents, analytics, and rule administration.
2. Why does it exist?	It turns backend evidence and workflow data into a navigable SOC user experience with role gates and auto-refreshing visual summaries.
3. What problem does it solve?	It gives analysts a controlled interface to submit logs, investigate evidence, rerun detection, train/predict ML, and manage incidents.
4. How does it interact with other modules?	AppRouter wraps protected pages in ProtectedRoute/AppShell and Users in AdminRoute. Pages call services; Dashboard uses React Query with 15-second refresh and Recharts.
5. What happens if removed?	The REST API remains available but the human SOC dashboard/demo workflow is lost.
6. Which files are responsible?	Primary responsibility is in `frontend/src/routes/AppRouter.tsx; frontend/src/features/auth/*; dashboard/pages/DashboardPage.tsx; logs/pages/UploadLogsPage.tsx; monitoring/pages/LiveMonitoringPage.tsx; detections/pages/ThreatDetectionPage.tsx; anomalies/pages/AnomalyDetectionPage.tsx; incidents/pages/IncidentManagementPage.tsx; analytics/pages/AnalyticsPage.tsx; rules/pages/DetectionRulesPage.tsx`.
7. Which APIs use it?	Pages consume their matching service routes; Dashboard consumes six analytics endpoints and renders summary cards/charts/recent events.
8. Which Mongo collections / ES indices?	Indirectly consumes Mongo/ES-backed API data; only direct client storage is the token wrapper.
9. What data enters?	User form actions, route state, API response objects, query filters, uploaded files, and auth context.
10. What data leaves?	Rendered pages, charts, tables, status controls, feedback states, and API mutations.

2.16 Docker Compose deployment

`docker-compose.yml; backend/Dockerfile; frontend/Dockerfile`

Question	Project-specific answer
1. What is this module?	Local multi-container deployment definition for MongoDB 7, Elasticsearch 8.15, optional Kibana, FastAPI backend, and Vite/React frontend.
2. Why does it exist?	The project requires two stateful services plus two applications; Compose makes versions, ports, health checks, mounts, and startup dependencies repeatable.
3. What problem does it solve?	It removes manual setup drift and gives an academic demo a predictable single command startup path.
4. How does it interact with other modules?	Backend receives Compose-specific Mongo/ES hostnames and mounts backend code/models/datasets/mappings/sample logs. Frontend receives proxy/base URL environment values and waits for backend.
5. What happens if removed?	The source can still run manually, but full-stack environment setup becomes error-prone; dependencies and persistent data are not orchestrated.
6. Which files are responsible?	Primary responsibility is in `docker-compose.yml; backend/Dockerfile; frontend/Dockerfile`.
7. Which APIs use it?	Exposes Mongo 27017, ES 9200, Kibana 5601, backend 8000, and frontend 5173; backend serves all API endpoints.

Question	Project-specific answer
8. Which Mongo collections / ES indices?	Named volumes *mongo_data* and *es_data*; mounted *ml/models*; corresponding Mongo collections and ES indices.
9. What data enters?	Images/build contexts, .env, Compose environment overrides, mapped source directories, and health-check conditions.
10. What data leaves?	Healthy interconnected containers, persistent volumes, host ports, and repeatable runtime topology.

3. Part B — Complete end-to-end workflow

This traces the implemented route from an authenticated analyst to dashboard analytics, including functions, storage operations, routes, and frontend response shapes.

3.1 Login and credential validation

The user submits *LoginForm* through *AuthContext.login* to *authService.login*, which POSTs */api/v1/auth/login*. In *backend/app/api/v1/auth.py::login*, Mongo *get_user_by_username* retrieves *users*; *verify_password* checks bcrypt; *create_access_token* and *create_refresh_token* sign HS256 JWT claims *sub*, *role*, *type*, *iat*, *exp*. Mongo updates *last_login*. The frontend receives *{access_token, refresh_token, token_type, user}*, saves tokens in local/session storage, and holds *UserPublic* in context.

3.2 JWT use, refresh, and RBAC

Every Axios request from *api/client.ts* attaches *Authorization: Bearer <access token>*. FastAPI *oauth2_scheme* extracts it; *get_current_user* calls *decode_token(expected_type='access')* and reloads the active user from Mongo. *require_role* adds precise 403 role gates. On a 401, the frontend serializes refresh calls with *refreshInFlight*, POSTs */auth/refresh*, saves the rotated token pair, and retries once; invalid refresh clears storage and navigates to */login*.

3.3 Upload and HTTP validation

UploadLogsPage calls *logService.upload(file, sourceType, sourceHost)*, creating multipart FormData. *POST /logs/upload* checks that extension is .log/.txt/.json, limits raw bytes using *MAX_UPLOAD_FILE_SIZE_MB*, requires UTF-8 text, and passes raw content plus the authenticated uploader name to *default_pipeline.process*. Programmatic sources can POST JSON to */logs/ingest* with source_type, source_host, and content.

3.4 Parser selection and raw extraction

In *IngestionPipeline.process*, *PARSER_REGISTRY* maps SourceType to a stateless parser. Linux extracts failed/accepted SSH and sudo patterns; Windows loads JSON array or NDJSON and maps Event IDs 4624/4625/4672/4720/4728; firewall extracts action/protocol/source/destination/port; web extracts Combined Log Format fields. Each produces ParsedEvent, preserving raw evidence. A structurally invalid Windows payload raises a controlled 422; individual unrecognized lines are skipped.

3.5 Normalize, validate, deduplicate, and index

For every ParsedEvent, *normalizer.normalize* produces NormalizedLog. It makes timestamps UTC, resolves valid severity hints or LOW default, and computes SHA-256 from source type, host, raw log, and timestamp. Pipeline queries ES *security-logs-** for existing log_ids across all daily indices, removes duplicates, then calls *bulk_index_logs* into today's *security-logs-YYYY.MM.DD*. It writes an IngestionBatch audit document to Mongo *ingestion_batches* and returns *{accepted, rejected, duplicates, errors, log_ids, batch_id}* to the frontend.

3.6 Rule engine and automatic AI scoring

Main registered *handle_ingested_logs* as the pipeline callback. It runs enabled rules over just-indexed NormalizedLog objects, then calls *default_prediction_service.predict*. Prediction safely returns an empty list if no trained model exists, so AI readiness never blocks rule detection. Combined DetectionEvents are persisted in Mongo *detections*. This common event schema keeps later processing independent of whether an event came from rule or ML.

3.7 Alerts and correlated incidents

generate_alerts creates Mongo *alerts* only for high/critical detections. Then *correlate_detections* derives a threat key (rule name or ml_anomaly) and searches for active Mongo incidents with the same entity value and correlation key. It adds evidence/detection IDs and escalates severity on a matching OPEN/INVESTIGATING incident; otherwise it atomically allocates an INC number from *counters* and creates a Mongo *incidents* document.

3.8 Dashboard, live monitoring, and analytics

DashboardPage uses React Query every 15 seconds to request */analytics/summary*, threat timeline, severity distribution, attack types, top IPs, and recent events. Analytics combines ES counts/aggregations with Mongo counts/aggregates. Live Monitoring calls */logs/search*, receiving *{total, results: NormalizedLog[]}*. Threat/incident pages receive *{detections, count}* and *{incidents, count}*; all have field names matching backend schemas rather than custom UI-only shapes.

End-to-end flow in one line:

```
Login → JWT → Upload → Parser → Normalize → ES → RuleEngine + AI → Mongo detections/alerts → Correlate incidents → Dashboard/Analytics
```

4. Part C — Feature deep-dives

Use these as standalone explanations during a code walkthrough. Each entry states purpose, problem/importance, what breaks without it, and the exact implementation files.

4.1 Authentication

Question	Project-specific answer
How in this project	JWT login is implemented, not mocked: bcrypt verifies a Mongo user, access and refresh JWTs carry subject/role/type, Axios retries one failed request after refresh, and FastAPI reloads the active user for each protected request.
Why / problem / importance / without it	It protects SOC data and permits role-specific admin actions. Without it, upload, rule changes, model training, and incident operations would be unauthenticated.
Files to cite	`backend/app/api/v1/auth.py; backend/app/core/security.py; backend/app/core/dependencies.py; frontend/src/api/client.ts; frontend/src/api/tokenStorage.ts; frontend/src/features/auth/context/AuthContext.tsx`

4.2 Dashboard

Question	Project-specific answer
How in this project	The dashboard is a live operational summary, not a static report: six React Query calls refresh every 15 seconds and Recharts renders trend, severity, attack type, and IP views.
Why / problem / importance / without it	It compresses thousands of records into SOC prioritization signals. Without it, an analyst must query separate endpoints and cannot rapidly assess risk.
Files to cite	`frontend/src/features/dashboard/pages/DashboardPage.tsx; backend/app/api/v1/analytics.py`

4.3 Log Upload

Question	Project-specific answer
How in this project	Multipart upload validates source type, extension, UTF-8, and maximum size before invoking one reusable pipeline. The response exposes accepted/rejected/duplicate counts and batch ID.
Why / problem / importance / without it	It is the controlled gateway for evidence. Without it, logs are neither normalized nor auditable nor automatically detected.
Files to cite	`frontend/src/services/logService.ts; backend/app/api/v1/logs.py; backend/app/ingestion/pipeline.py`

4.4 Parsing

Question	Project-specific answer
How in this project	Four source-specific strategies understand Linux sshd/sudo, Windows exported Security JSON, firewall ALLOW/DENY lines, and Apache/Nginx Combined Log Format.
Why / problem / importance / without it	It extracts meaningful fields without conflating source-format handling with security verdicts. Without it, raw text cannot feed generic downstream rules.
Files to cite	`backend/app/ingestion/parsers/*.py`

4.5 Normalization

Question	Project-specific answer
How in this project	Parsed events become NormalizedLog records with UTC timestamps, canonical enums, raw evidence, parsed fields, and deterministic SHA-256 IDs.

Question	Project-specific answer
Why / problem / importance / without it	It is the contract that makes all downstream logic source-agnostic and idempotent. Without it, every rule/ML feature would be parser-specific and duplicates would recur.
Files to cite	`backend/app/ingestion/normalizer.py; backend/app/models/log_schema.py`

4.6 Windows, Linux, Firewall, and Web processing

Question	Project-specific answer
How in this project	Windows maps Event IDs to login/privilege semantics; Linux extracts auth and sudo; firewall maps DENY connections and ports; web extracts request paths for payload scanning.
Why / problem / importance / without it	Each produces security-relevant fields unique to its source. Without a parser, that source type is unsupported but the architecture remains extensible through BaseLogParser and PARSER_REGISTRY.
Files to cite	`backend/app/ingestion/parsers/windows_parser.py; linux_parser.py; firewall_parser.py; web_parser.py`

4.7 Rule detection

Question	Project-specific answer
How in this project	Eight explainable rules emit MITRE-mapped DetectionEvents, including brute force, port scanning, SQLi, XSS, traversal, privilege escalation, and suspicious privileged login.
Why / problem / importance / without it	Known attack patterns need deterministic, reviewer-friendly detection. Without rules, training-independent signature coverage disappears.
Files to cite	`backend/app/detection/engine.py; backend/app/detection/rules/*.py`

4.8 AI anomaly detection

Question	Project-specific answer
How in this project	Isolation Forest models eight source-IP features, requires at least five IP entities, versions artifacts, and turns anomalies into the same DetectionEvent shape as rules.
Why / problem / importance / without it	It supplements signatures for unusual patterns. Without it, the application remains operational but cannot surface statistical outliers or AI metrics.
Files to cite	`backend/app/detection/ml/*.py; ml/models/registry_index.json`

4.9 Incidents

Question	Project-specific answer
How in this project	Correlation reuses active incidents by entity plus threat key, accumulates evidence, escalates severity, and service code enforces a state machine with notes and assignment.
Why / problem / importance / without it	It converts noisy technical findings into auditable analyst work. Without it, detections are isolated and there is no investigation workflow.
Files to cite	`backend/app/incidents/correlator.py; backend/app/incidents/service.py; backend/app/models/incident_schema.py`

4.10 Alerts

Question	Project-specific answer
How in this project	High/critical detections create in-application Mongo Alert records. External dispatch is intentionally a future seam, represented by channel_dispatched rather than claimed as implemented.
Why / problem / importance / without it	It provides urgency filtering. Without it, detections still exist, but critical alert count/feed disappears.

Question	Project-specific answer
Files to cite	`backend/app/detection/alerting.py; backend/app/models/alert_schema.py`

4.11 Analytics

Question	Project-specific answer
How in this project	Analytics safely combines ES telemetry aggregations with Mongo operational counts. ES failures return defaults so a partially unavailable search cluster does not crash the dashboard endpoint.
Why / problem / importance / without it	It produces decision-ready measures. Without it, frontend charts/cards have no composed data source.
Files to cite	`backend/app/api/v1/analytics.py; backend/app/db/elastic.py; backend/app/db/mongo.py`

4.12 Elasticsearch

Question	Project-specific answer
How in this project	Daily security-logs indices retain normalized evidence and support full-text raw-log search, filters, time queries, duplicate checks, and aggregations.
Why / problem / importance / without it	Search-oriented, time-indexed telemetry needs different access patterns from user/incident documents. Without ES, ingestion and evidence monitoring fail.
Files to cite	`backend/app/db/elastic.py; elasticsearch/mappings/security_logs_mapping.json`

4.13 MongoDB

Question	Project-specific answer
How in this project	Mongo holds users, workflow records, rule configuration, detections, alerts, and batch audit records with indexes for correctness/performance.
Why / problem / importance / without it	It persists mutable application state and incident history. Without it, authentication, incident management, and dashboard counters fail.
Files to cite	`backend/app/db/mongo.py`

4.14 Docker

Question	Project-specific answer
How in this project	Compose brings up Mongo, ES, optional Kibana, backend, and frontend with health checks, networks/hostnames, mounts, and persistent volumes.
Why / problem / importance / without it	It makes a five-service demo reproducible. Without it, manual setup is possible but brittle.
Files to cite	`docker-compose.yml; backend/Dockerfile; frontend/Dockerfile`

4.15 React frontend

Question	Project-specific answer
How in this project	Routes are protected by auth context; services isolate HTTP; page features render upload, monitoring, detection, anomaly, incident, rule, and analytics workflows.
Why / problem / importance / without it	It makes the backend usable by an analyst and demonstrable in a viva. Without it, only Swagger/API tooling remains.
Files to cite	`frontend/src/routes/AppRouter.tsx; frontend/src/features/*; frontend/src/services/*`

5. Part C.1 — Parser dossiers

Each parser is documented with the same ten-question developer template. These describe implemented source adapters, not generic log-parser claims.

WindowsEventLogParser

`backend/app/ingestion/parsers/windows_parser.py`

Question	Project-specific answer
1. What is this?	**WindowsEventLogParser** is a concrete implementation unit in `backend/app/ingestion/parsers/windows_parser.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Windows Security exports as a JSON array, object, or Winlogbeat-style NDJSON.
4. How does it work?	parse → _load_records → _parse_record → _parse_timestamp. It maps 4624/4625/4672/4720/4728 to successful_login, failed_login, privileged_logon, account_created, and group_membership_added; unknown IDs become windows_event.
5. What uses it?	WindowsFailedLoginRule and PrivilegeEscalationRule consume the normalized event types.
6. What leaves?	TargetUserName, IpAddress and TimeCreated become user, source_ip and timestamp; remaining fields and event_id remain parsed_fields.
7. What boundary matters?	This is JSON-export ingestion, not a binary EVTX reader. Invalid NDJSON lines are skipped; entirely unparseable content raises LogParsingError.
8. Where in the call chain?	Windows records → ParsedEvent → normalize → NormalizedLog → ES security-logs-YYYY.MM.DD.
9. What must you say in a viva?	PARSER_REGISTRY selects it for SourceType.WINDOWS. It extracts evidence only; it does not declare an attack.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

LinuxAuthLogParser

`backend/app/ingestion/parsers/linux_parser.py`

Question	Project-specific answer
1. What is this?	**LinuxAuthLogParser** is a concrete implementation unit in `backend/app/ingestion/parsers/linux_parser.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Linux auth.log/syslog lines for sshd failed/accepted passwords and sudo.
4. How does it work?	parse → _parse_line → _build_login_event or _build_sudo_event. It yields failed_login, successful_login, or privilege_command and records host/port or target_user/command.
5. What uses it?	SSHBruteForceRule, PrivilegeEscalationRule, and SuspiciousAdminLoginRule consume these event types.
6. What leaves?	The syslog timestamp has no year, so _parse_syslog_timestamp assumes the current UTC year; normalize then supplies canonical UTC.
7. What boundary matters?	It intentionally ignores unrelated syslog services; this is an auth parser, not a complete Linux logging parser.
8. Where in the call chain?	Raw lines → ParsedEvent → normalize → ES, detection, ML features.
9. What must you say in a viva?	SSH detection is four failures in 120 seconds. Parser severity hints describe records, while rules produce verdict severity.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

FirewallLogParser

`backend/app/ingestion/parsers/firewall_parser.py`

Question	Project-specific answer
1. What is this?	**FirewallLogParser** is a concrete implementation unit in `backend/app/ingestion/parsers/firewall_parser.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Project ALLOW/DENY connection lines with source/destination address and port.
4. How does it work?	A compiled _FIREWALL_LINE_RE extracts timestamp, action, protocol, addresses, ports and flags. DENY maps to connection_denied; ALLOW maps to connection_allowed.
5. What uses it?	PortScanRule consumes denied records with source_ip, dest_ip and dest_port.
6. What leaves?	protocol, src_port, action and flags stay in parsed_fields; dest_port is promoted to NormalizedLog.
7. What boundary matters?	It is a sample-format adapter, not a universal pfSense/Cisco/iptables parser.
8. Where in the call chain?	Connection text → ParsedEvent → normalize → ES → PortScanRule.
9. What must you say in a viva?	The parser does not call a denial malicious; scanning policy is in PortScanRule.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

WebServerLogParser

`backend/app/ingestion/parsers/web_parser.py`

Question	Project-specific answer
1. What is this?	**WebServerLogParser** is a concrete implementation unit in `backend/app/ingestion/parsers/web_parser.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Apache/Nginx Combined Log Format access lines.
4. How does it work?	parse applies _COMBINED_LOG_RE, parses %d/%b/%Y:%H:%M:%S %z, and emits http_request records with method/path/status/response size/referrer/user-agent.
5. What uses it?	SQLInjectionRule, XSSRule, DirectoryTraversalRule, and ML avg_response_size_bytes feature extraction.
6. What leaves?	IP is source_ip; path and response_size_bytes are retained in parsed_fields for downstream use.
7. What boundary matters?	Malformed Combined-format lines are skipped; custom JSON access logs require another adapter.
8. Where in the call chain?	Access line → ParsedEvent → normalize → ES → payload rules/ML.
9. What must you say in a viva?	It does extraction only. SQLi/XSS/traversal classification is deliberately deferred to detection rules.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

6. Part C.2 — Detection-rule dossiers

Each of the eight registered rule classes receives its own implementation-grounded explanation. Rule evaluation is invoked through RuleEngine.run, never directly by the frontend.

SSHBruteForceRule

`backend/app/detection/rules/ssh_brute_force.py`

Question	Project-specific answer
1. What is this?	**SSHBruteForceRule** is a concrete implementation unit in `backend/app/detection/rules/ssh_brute_force.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Linux failed_login records grouped by source IP.
4. How does it work?	DEFAULT_THRESHOLD=4 and DEFAULT_WINDOW_SECONDS=120; group_by_source_ip and window_seconds permit a longer window only at double volume.
5. What uses it?	RuleEngine.run returns its DetectionEvent to handle_ingested_logs.
6. What leaves?	T1110; confidence rises from 0.5, severity is HIGH or CRITICAL at three times threshold, and all group log IDs are evidence.
7. What boundary matters?	It sees Linux failed_login only; it does not prove compromise.
8. Where in the call chain?	Normalized logs → RuleEngine.run → Mongo detections → generate_alerts → correlate_detections.
9. What must you say in a viva?	RuleConfig may tune threshold/window; without it Linux failures remain searchable but produce no brute-force detection.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

WindowsFailedLoginRule

`backend/app/detection/rules/windows_failed_login.py`

Question	Project-specific answer
1. What is this?	**WindowsFailedLoginRule** is a concrete implementation unit in `backend/app/detection/rules/windows_failed_login.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Windows failed_login (mapped Event ID 4625) grouped by source IP.
4. How does it work?	Threshold 3 and 120-second configuration; targets and observed span are described in its event.
5. What uses it?	RuleEngine.run invokes it after enabled checking.
6. What leaves?	T1110.001; HIGH normally and CRITICAL at triple threshold; evidence IDs are preserved.
7. What boundary matters?	It is deliberately separate from SSH because Windows tuning/semantics differ.
8. Where in the call chain?	Normalized Windows logs → engine → shared detection/alert/incident flow.
9. What must you say in a viva?	It is behavioral evidence, not a parser claim that every 4625 is malicious.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

PortScanRule

`backend/app/detection/rules/port_scan.py`

Question	Project-specific answer
1. What is this?	**PortScanRule** is a concrete implementation unit in `backend/app/detection/rules/port_scan.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Denied firewall connections having source IP, destination IP, and destination port.
4. How does it work?	Groups by (source_ip, dest_ip), counts distinct ports, applies threshold 5 and 60-second window.
5. What uses it?	RuleEngine.run; no route or database is accessed directly by the rule.
6. What leaves?	T1046; MEDIUM becomes HIGH at double threshold; explanation lists scanned ports and span.
7. What boundary matters?	Only connection_denied firewall records qualify.
8. Where in the call chain?	Normalized firewall evidence → engine → Mongo detections/alerts/incidents.
9. What must you say in a viva?	It signals reconnaissance consistent with scanning, not demonstrated service compromise.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

SQLInjectionRule

`backend/app/detection/rules/sql_injection.py`

Question	Project-specific answer
1. What is this?	**SQLInjectionRule** is a concrete implementation unit in `backend/app/detection/rules/sql_injection.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Web parsed_fields.path or raw_log.
4. How does it work?	find_matching_pattern searches tautologies, UNION SELECT, comments, stacked writes, SLEEP and information_schema.
5. What uses it?	RuleEngine.run and the shared post-ingest handler.
6. What leaves?	T1190; confidence .75, UNION is CRITICAL and other signatures HIGH.
7. What boundary matters?	It evaluates web records only and is evadable by novel obfuscation.
8. Where in the call chain?	Web log → normalized path → rule → DetectionEvent with one evidence ID.
9. What must you say in a viva?	It records a finding; it does not block requests or dispatch external alerts.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

XSSRule

`backend/app/detection/rules/xss.py`

Question	Project-specific answer
1. What is this?	**XSSRule** is a concrete implementation unit in `backend/app/detection/rules/xss.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Web request paths/query strings.
4. How does it work?	find_matching_pattern checks script tags, event-handler injection, javascript URIs, image tags, document cookie/location and iframe.
5. What uses it?	RuleEngine.run.
6. What leaves?	T1059.007; confidence .7, MEDIUM, source IP and matching evidence ID.
7. What boundary matters?	It is a transparent signature detector, not a complete browser exploit analysis.
8. Where in the call chain?	Web evidence → normalized path → XSS DetectionEvent.
9. What must you say in a viva?	Removing it preserves logs but removes explicit XSS detections.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

DirectoryTraversalRule

`backend/app/detection/rules/directory_traversal.py`

Question	Project-specific answer
1. What is this?	**DirectoryTraversalRule** is a concrete implementation unit in `backend/app/detection/rules/directory_traversal.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Web request paths.
4. How does it work?	Matches repeated ../, encoded traversal, and sensitive targets including etc/passwd, boot.ini, win.ini and system32.
5. What uses it?	RuleEngine.run.
6. What leaves?	T1083; confidence .8, HIGH severity, one evidence log ID.
7. What boundary matters?	It detects attempted traversal, not confirmed file disclosure.
8. Where in the call chain?	Web evidence → normalized path → traversal DetectionEvent.
9. What must you say in a viva?	There is no threshold because this is a per-request signature.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

PrivilegeEscalationRule

`backend/app/detection/rules/privilege_escalation.py`

Question	Project-specific answer
1. What is this?	**PrivilegeEscalationRule** is a concrete implementation unit in `backend/app/detection/rules/privilege_escalation.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Linux privilege_command and Windows account_created/group_membership_added records.
4. How does it work?	evaluate dispatches to _evaluate_linux_sudo or _evaluate_windows_event; Linux requires target_user root.
5. What uses it?	RuleEngine.run.
6. What leaves?	T1548; root sudo MEDIUM/.55, Windows account creation MEDIUM, group membership HIGH/.6.
7. What boundary matters?	Root sudo may be valid maintenance and the explanation explicitly asks for contextual review.
8. Where in the call chain?	OS evidence → rule → user-entity DetectionEvent.
9. What must you say in a viva?	The code makes no automatic remediation decision.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

SuspiciousAdminLoginRule

`backend/app/detection/rules/suspicious_admin_login.py`

Question	Project-specific answer
1. What is this?	**SuspiciousAdminLoginRule** is a concrete implementation unit in `backend/app/detection/rules/suspicious_admin_login.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	Successful Linux/Windows logins for root, admin, administrator, or configured equivalents.
4. How does it work?	Checks privileged_accounts and 06:00–22:00 UTC defaults against timestamp hour.
5. What uses it?	RuleEngine.run.
6. What leaves?	T1078; MEDIUM/.4 and explanation explicitly calls it a low-confidence signal.
7. What boundary matters?	It only qualifies off-hours successful logins for named privileged accounts.
8. Where in the call chain?	Normalized auth evidence → rule → user-entity DetectionEvent.
9. What must you say in a viva?	RuleConfig can tune accounts/hours; corroborate rather than treating it as proof.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

7. Part C.3 — Complete API and frontend walkthrough

The following pages name every backend router route and every routed feature page. They make the REST-to-React contract explicit.

API router: Authentication

`backend/app/api/v1/auth.py`

Question	Project-specific answer
1. What is this?	**API router: Authentication** is a concrete implementation unit in `backend/app/api/v1/auth.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	POST /api/v1/auth/login; POST /api/v1/auth/refresh; POST /api/v1/auth/register; GET /api/v1/auth/me.
4. How does it work?	login uses get_user_by_username, verify_password, create_access_token, create_refresh_token and update_last_login; register is admin-gated.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/auth.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: users collection; no ES responsibility.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: Logs

`backend/app/api/v1/logs.py`

Question	Project-specific answer
1. What is this?	**API router: Logs** is a concrete implementation unit in `backend/app/api/v1/logs.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	POST /api/v1/logs/upload; POST /api/v1/logs/ingest; GET /api/v1/logs/search; GET /api/v1/logs/batches.
4. How does it work?	upload/ingest call default_pipeline.process; search queries ES; batches reads Mongo audit data.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/logs.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: security-logs-* evidence and ingestion_batches audit documents.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: Detections

`backend/app/api/v1/detections.py`

Question	Project-specific answer
1. What is this?	**API router: Detections** is a concrete implementation unit in `backend/app/api/v1/detections.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	GET /api/v1/detections; GET /api/v1/detections/alerts; GET /api/v1/detections/rules; POST /api/v1/detections/run.
4. How does it work?	Lists persisted results, exposes rules, and supports a manual detection run in addition to automatic ingestion callback detection.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/detections.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: detections and alerts collections, linked ES evidence IDs.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: ML

`backend/app/api/v1/ml.py`

Question	Project-specific answer
1. What is this?	**API router: ML** is a concrete implementation unit in `backend/app/api/v1/ml.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	POST /api/v1/ml/train; POST /api/v1/ml/predict; GET /api/v1/ml/models; GET /api/v1/ml/models/latest; POST /api/v1/ml/evaluate.
4. How does it work?	Uses TrainingService, PredictionService and ModelRegistry; train/evaluate are privileged operations.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/ml.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: ES history, filesystem model artifacts/registry_index.json, then Mongo detection workflow.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: Incidents

`backend/app/api/v1/incidents.py`

Question	Project-specific answer
1. What is this?	**API router: Incidents** is a concrete implementation unit in `backend/app/api/v1/incidents.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	GET /api/v1/incidents; GET /api/v1/incidents/{incident_ref}; PATCH /api/v1/incidents/{incident_ref}/status; POST /api/v1/incidents/{incident_ref}/notes; PATCH /api/v1/incidents/{incident_ref}/assign.
4. How does it work?	Delegates to incidents/service.py, which validates transitions and writes actor-attributed notes.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/incidents.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: incidents and counters.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: Analytics

`backend/app/api/v1/analytics.py`

Question	Project-specific answer
1. What is this?	**API router: Analytics** is a concrete implementation unit in `backend/app/api/v1/analytics.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	GET /api/v1/analytics/summary, /top-ips, /severity-distribution, /threat-timeline, /attack-types, /recent-events.
4. How does it work?	Composes dashboard-shaped data for DashboardPage and AnalyticsPage.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/analytics.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: ES telemetry aggregations plus Mongo operational counts.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

API router: Rules

`backend/app/api/v1/rules.py`

Question	Project-specific answer
1. What is this?	**API router: Rules** is a concrete implementation unit in `backend/app/api/v1/rules.py`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	GET /api/v1/rules; PATCH /api/v1/rules/{rule_name}.
4. How does it work?	Exposes and updates active/persisted detection policy; changes require admin protection.
5. What uses it?	The corresponding frontend service and authenticated FastAPI router callers.
6. What leaves?	Validated JSON / response models or HTTP error responses.
7. What boundary matters?	The router is HTTP orchestration; business work stays in services/pipeline.
8. Where in the call chain?	/api/v1 → backend/app/api/v1/rules.py → service/repository → response.
9. What must you say in a viva?	Storage responsibility: detection_rules collection and RuleEngine instances.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: AuthContext

`frontend/src/features/auth/context/AuthContext.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: AuthContext** is a concrete implementation unit in `frontend/src/features/auth/context/AuthContext.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Owns session/user state and delegates credentials to authService; token storage/client refresh are centralized rather than reimplemented per page.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → AuthContext → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: AppRouter

`frontend/src/routes/AppRouter.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: AppRouter** is a concrete implementation unit in `frontend/src/routes/AppRouter.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Maps protected UI routes and route guards to the application feature pages.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → AppRouter → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: DashboardPage

`frontend/src/features/dashboard/pages/DashboardPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: DashboardPage** is a concrete implementation unit in `frontend/src/features/dashboard/pages/DashboardPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses analyticsService to render summary, trend, severity, attack, IP and recent-event views.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → DashboardPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: UploadLogsPage

`frontend/src/features/logs/pages/UploadLogsPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: UploadLogsPage** is a concrete implementation unit in `frontend/src/features/logs/pages/UploadLogsPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses logService to submit source/file metadata and display pipeline counts/batch results.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → UploadLogsPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: LiveMonitoringPage

`frontend/src/features/monitoring/pages/LiveMonitoringPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: LiveMonitoringPage** is a concrete implementation unit in `frontend/src/features/monitoring/pages/LiveMonitoringPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses logService search to investigate normalized, ES-backed evidence.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → LiveMonitoringPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: ThreatDetectionPage

`frontend/src/features/detections/pages/ThreatDetectionPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: ThreatDetectionPage** is a concrete implementation unit in `frontend/src/features/detections/pages/ThreatDetectionPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses detectionService for detections, alerts, rule metadata and manual run actions.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → ThreatDetectionPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: AnomalyDetectionPage

`frontend/src/features/anomalies/pages/AnomalyDetectionPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: AnomalyDetectionPage** is a concrete implementation unit in `frontend/src/features/anomalies/pages/AnomalyDetectionPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses ML-facing services for training, prediction, versions and evaluation.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → AnomalyDetectionPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: IncidentManagementPage

`frontend/src/features/incidents/pages/IncidentManagementPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: IncidentManagementPage** is a concrete implementation unit in `frontend/src/features/incidents/pages/IncidentManagementPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses incidentService for filtering, transitions, notes and assignment.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → IncidentManagementPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: AnalyticsPage

`frontend/src/features/analytics/pages/AnalyticsPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: AnalyticsPage** is a concrete implementation unit in `frontend/src/features/analytics/pages/AnalyticsPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses analyticsService for deeper API-backed aggregation views.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → AnalyticsPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: DetectionRulesPage

`frontend/src/features/rules/pages/DetectionRulesPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: DetectionRulesPage** is a concrete implementation unit in `frontend/src/features/rules/pages/DetectionRulesPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Uses ruleService for inspection/administration of rule policy.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → DetectionRulesPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

Frontend: SettingsPage and UserManagementPage

`frontend/src/features/settings/pages/SettingsPage.tsx; frontend/src/features/users/pages/UserManagementPage.tsx`

Question	Project-specific answer
1. What is this?	**Frontend: SettingsPage and UserManagementPage** is a concrete implementation unit in `frontend/src/features/settings/pages/SettingsPage.tsx; frontend/src/features/users/pages/UserManagementPage.tsx`.
2. Why does it exist?	It keeps this responsibility separate from HTTP controllers, storage repositories, and presentation code.
3. What enters?	User interaction, route state, and typed service/API data.
4. How does it work?	Remaining routed UI surfaces that should be acknowledged in a complete frontend walkthrough.
5. What uses it?	React route layout, AuthContext, and the named frontend service layer.
6. What leaves?	Rendered controls, tables, charts, upload feedback, or analyst workflow state.
7. What boundary matters?	This code never accesses MongoDB or Elasticsearch directly; it uses authenticated REST services.
8. Where in the call chain?	AppRouter → SettingsPage and UserManagementPage → authService/logService/detectionService/incidentService/analyticsService/ruleService as applicable → /api/v1.
9. What must you say in a viva?	The exact matching service depends on the page; keep token refresh and RBAC on the shared client/backend boundary.
10. What breaks if removed?	The surrounding platform may still store raw evidence, but this specific transformation or detection is no longer available.

5. Part D — Viva learning guide

5.1 Beginner explanation

This project is a lightweight SIEM-style platform. A user logs in, uploads security logs from Linux, Windows, a firewall, or a web server, and the backend converts unlike source formats into one normalized record. Elasticsearch keeps the searchable raw evidence. Detection rules and an optional AI model turn suspicious behavior into findings. High-severity findings create alerts, related findings become incidents, and the React dashboard presents the operational picture. MongoDB stores the users and the workflow/history data.

5.2 Intermediate explanation

The design uses boundaries. FastAPI routers handle HTTP and authorization; the pipeline orchestrates ingestion; parser strategies handle source formats; the normalizer enforces the canonical contract; repositories isolate ES and Mongo; BaseRule and AnomalyModel make detection extensible; the correlator turns events into analyst work. This reduces coupling: adding a parser or rule should not require rewriting controllers, dashboards, or incident logic.

5.3 Technical explanation

At startup, lifespan connects Motor, ensures unique/query indexes, connects AsyncElasticsearch/template, restores persisted rule enabled state, and injects the async post-ingest callback. Deterministic SHA-256 IDs plus cross-daily-index ES search implement idempotence. The callback applies fault isolation: rule failures are isolated per rule; ML failure is caught separately; correlation failure cannot roll back indexed logs. ML trains an IsolationForest on a per-IP feature matrix and persists versioned joblib artifacts with JSON metadata.

5.4 How to present each module

- Start at main.py: explain lifecycle, router mounting, callback wiring, then move inward.
- For auth, show login → JWT claims → Axios Bearer interceptor → get_current_user/RBAC.
- For ingestion, emphasize the deliberate parser / normalizer / pipeline separation and deterministic IDs.
- For data stores, say ES is evidence/search/aggregation; Mongo is identity/workflow/config/audit.
- For detection, show a concrete Linux SSH or web SQLi example and name the emitted DetectionEvent fields.
- For AI, explain it as a complementary outlier detector, state the eight features, five-entity minimum, version registry, and batch-relative risk limitation honestly.
- For incidents, demonstrate correlation key + entity grouping and the no-delete state machine.
- For UI, show service layer and dashboard React Query refresh rather than describing React only in general terms.

5.5 What to say during the demo

- Open the dashboard and explain that cards and charts are fed by /analytics endpoints combining ES and Mongo—not hard-coded UI numbers.
- Log in with the seeded admin only in a controlled demo; state that default credentials must be changed in any shared deployment.
- Upload sample_logs/linux_auth.log as linux. Point out accepted/duplicate counts and the ingestion batch audit trail.
- Open Threat Detection or Dashboard recent events. Explain SSHBruteForceRule grouping by source IP and the MITRE T1110 mapping.
- Open Incidents. Show INC reference, evidence log IDs, assignment, a note, and a legal status transition.
- Upload web_access.log as web_server and explain SQL injection, XSS, and traversal are detected after parsing, not classified by the parser.
- Optionally train/predict only after enough distinct IPs exist. Explain no trained model means automatic ML scoring safely returns no result rather than failing ingestion.

- End at Docker Compose: demonstrate that Mongo and Elasticsearch have persistent volumes and backend waits for health checks.

5.6 Most important parts to understand

- The complete ingestion chain and deterministic deduplication.
- Why the two databases have different roles.
- The DetectionEvent contract unifies rule and ML outputs.
- RBAC and the frontend token refresh path.
- Correlation keys, atomic incident sequence, severity escalation, and status transition rules.
- The ML model's honest scope: Isolation Forest is unsupervised, risk is batch-relative, and evaluation uses labeled synthetic data.

5.7 Files to study first, in order

Question	Project-specific answer
1. backend/app/main.py	The composition root reveals runtime lifecycle, router map, and ingestion-to-detection wiring.
2. backend/app/ingestion/pipeline.py	It is the central operational flow from upload to indexed/audited/detected data.
3. backend/app/ingestion/normalizer.py and models/log_schema.py	They define the canonical data contract and idempotent log ID.
4. backend/app/db/elastic.py and db/mongo.py	They explain the split storage architecture and exact collections/indices.
5. backend/app/detection/engine.py and rules/base_rule.py	They explain pluggable, fault-isolated rule execution.
6. One threshold rule and one web rule	Study ssh_brute_force.py and sql_injection.py to present both behavioral and signature styles.
7. backend/app/detection/ml/feature_extraction.py, training_service.py, prediction_service.py	These are the technical AI story: features, fit, versioning, scoring, limitations.
8. backend/app/incidents/correlator.py and service.py	They show how detections become managed SOC cases.
9. backend/app/api/v1/auth.py and core/security.py/dependencies.py	They demonstrate authentication, JWT, and RBAC.
10. frontend/src/api/client.ts, services/analyticsService.ts, features/dashboard/pages/DashboardPage.tsx	They demonstrate real frontend-to-backend integration and live rendering.
11. docker-compose.yml	It closes the demo with operational reproducibility.

5.8 Expected viva questions and strong answers

Q1. Why use both MongoDB and Elasticsearch?

Strong answer: They have different responsibilities. Elasticsearch stores and searches time-oriented normalized raw logs in daily indices; MongoDB stores operational documents such as users, ingestion batches, detections, alerts, incidents, and rule state. Using one for both would either weaken evidence search or complicate mutable workflow data.

Q2. Explain the exact upload flow.

Strong answer: The browser sends multipart data to POST /logs/upload. The API checks extension, size, and UTF-8. IngestionPipeline selects a source parser, creates ParsedEvent objects, normalizes them to NormalizedLog with deterministic IDs, checks ES for duplicates across daily indices, bulk indexes new logs, saves a Mongo ingestion batch, then invokes the post-ingest detection callback.

Q3. How is duplicate ingestion prevented?

Strong answer: normalize.compute_log_id calculates SHA-256 from source type, source host, raw log text, and normalized timestamp. Pipeline queries security-logs-* for those IDs before bulk indexing. The cross-index search matters because the same log reuploaded on another day might otherwise target a different daily ES index.

Q4. Why are parser and normalizer separated?

Strong answer: Parsers answer source-specific extraction questions; normalizer owns canonical schema mapping, UTC treatment, severity fallback, and ID construction. That single-responsibility boundary means adding a log source does not require modifying every detection consumer.

Q5. How does RBAC work?

Strong answer: Login signs a JWT with username and role. Every protected endpoint runs get_current_user, which validates an access token then reloads the user from Mongo. require_role compares the current role with explicitly allowed roles; for example model training and rule updates are admin-only.

Q6. How do rules avoid one failure stopping all detection?

Strong answer: RuleEngine.run wraps each enabled rule evaluation in its own try/except and logs a failure before continuing. The post-ingestion handler separately wraps ML prediction and incident correlation, so a trained-model or correlator issue does not erase successful rule detection or indexed evidence.

Q7. Give an example rule threshold.

Strong answer: SSHBruteForceRule groups Linux failed_login records by source IP. It requires at least four attempts and expects a 120-second burst unless volume is at least double threshold; severity becomes critical at three times threshold. It records all evidence log IDs and MITRE T1110.

Q8. How is AI different from a rule here?

Strong answer: Rules have coded predicates and thresholds, so they detect known behaviors transparently. The Isolation Forest learns a statistical boundary over eight per-IP features and flags outlier entities. Both emit DetectionEvent, so alerting and incident correlation do not care which engine originated the finding.

Q9. What are the ML features?

Strong answer: failed_login_count, successful_login_count, total_event_count, distinct_event_types, distinct_dest_ports, distinct_dest_ips, average HTTP response size, and off-hours event ratio. Features are calculated per source IP, not per individual log.

Q10. What does ML risk percentage mean?

Strong answer: It is not a globally calibrated probability. PredictionService min-max normalizes decision scores within the current scored batch, with lower Isolation Forest score becoming higher 0–100 risk. The correct viva answer is to state this limitation and describe fixed-reference calibration as a future improvement.

Q11. How are incidents correlated?

Strong answer: The correlator derives an entity value and correlation key—usually the rule name, or ml_anomaly for ML—and looks for an OPEN or INVESTIGATING Mongo incident with the same pair. It adds unique detection/evidence IDs, updates timestamp, escalates severity if needed, or allocates a new INC- number atomically.

Q12. Why are incidents never deleted?

Strong answer: The project preserves security audit integrity. Analysts transition OPEN to INVESTIGATING, RESOLVED, and CLOSED; each change records a note and actor. Deleting an incident would remove investigative evidence and weaken accountability.

Q13. What happens if ES is unavailable at startup?

Strong answer: main.py logs the connection failure rather than immediately crashing the API, which helps Compose race conditions. Readiness reports degraded because it pings both Mongo and ES. Actual ingestion/search operations still require ES, so this is graceful startup visibility rather than a claim of offline ingestion.

Q14. What security limitations would you improve for production?

Strong answer: Replace default JWT secret with secret management and key rotation, use HTTPS, add rate limiting and token revocation, reduce browser token exposure with hardened cookie/session design, validate file content beyond extension, add queue-based ingestion/training, enforce ES auth/TLS, and use background workers plus monitoring.

Q15. Why use Docker Compose?

Strong answer: Compose provides a repeatable topology with health-gated Mongo/ES dependencies, named persistent volumes, mounts for model artifacts/datasets/log fixtures, service hostnames, and fixed demo ports. It prevents environment mismatch during a viva.

Q16. Which parser supports exported Windows data?

Strong answer: WindowsEventLogParser in ingestion/parsers/windows_parser.py accepts JSON arrays, a JSON object, and NDJSON. It maps selected Security Event IDs but does not parse binary .evtx files.

Q17. What exactly does the Linux parser ignore?

Strong answer: LinuxAuthLogParser only handles sshd failed/accepted-password and sudo patterns. Other syslog lines are intentionally skipped so an unrelated subsystem does not become a parser failure.

Q18. Why does the web parser not flag SQL injection itself?

Strong answer: WebServerLogParser extracts the Combined Log Format path and evidence. SQLInjectionRule performs the policy decision later, preserving parser/detection separation and independent tuning.

Q19. What makes an alert?

Strong answer: generate_alerts in detection/alerting.py persists AlertInDB records only for high or critical DetectionEvents. Low and medium detections remain stored detections without an alert.

Q20. Does this system send email or Telegram alerts?

Strong answer: No. alerting.py creates in-application Mongo alerts. Its comments identify external dispatch as out of scope, and the alerts/ dispatcher area should not be presented as implemented.

Q21. What is the incident status machine?

Strong answer: incidents/service.py allows OPEN to INVESTIGATING, RESOLVED, or CLOSED; INVESTIGATING to RESOLVED, CLOSED, or OPEN; RESOLVED to CLOSED or INVESTIGATING; CLOSED has no outgoing transition.

Q22. What occurs when an analyst assigns an open incident?

Strong answer: assign_analyst writes assigned_to, adds an audit note, and automatically changes OPEN to INVESTIGATING. It leaves other current statuses unchanged.

Q23. How is incident severity escalated?

Strong answer: correlator._upsert_for_detection compares ranks low/medium/high/critical. A higher related detection updates the active incident and appends a system investigation note.

Q24. What does IsolationForestModel contribute?

Strong answer: IsolationForestModel is the concrete AnomalyModel implementation fitted by TrainingService. It is an unsupervised outlier detector, so it complements explicit rule predicates rather than replacing them.

Q25. How is a trained model versioned?

Strong answer: ModelRegistry.register_model writes a new isolation_forest_vN.joblib artifact and appends metadata to registry_index.json. Retraining does not overwrite the currently saved version.

Q26. Why are five entities required for training?

Strong answer: TrainingService rejects fewer than MIN_TRAINING_SAMPLES=5 distinct source IPs because a model trained on only a few entities has an unhelpful notion of normality.

Q27. What happens when no ML model is trained?

Strong answer: PredictionService.predict returns an empty list and logs a warning. This means automatic ingestion still runs rules and indexing successfully.

Q28. What is the frontend authentication chain?

Strong answer: LoginPage calls AuthContext, AuthContext calls authService, the shared Axios client stores/attaches tokens, and a single refresh-in-flight path calls /auth/refresh on a 401 before one retry.

Q29. Which data belongs in Mongo versus Elasticsearch?

Strong answer: Mongo holds users, detections, alerts, incidents, ingestion_batches, detection_rules, counters and operational documents. ES holds time-oriented normalized log evidence in security-logs-YYYY.MM.DD searched as security-logs-*.

Q30. How can a manual rule run differ from upload?

Strong answer: POST /api/v1/detections/run is an explicit controller action over retrieved logs. The normal upload path calls IngestionPipeline.process and then automatically reaches handle_ingested_logs for fresh indexed records.

6. Appendix — API map, data stores, and demo checklist

6.1 API map

Question	Project-specific answer
Authentication	POST /api/v1/auth/login; POST /auth/refresh; POST /auth/register (admin); GET /auth/me
Logs	POST /api/v1/logs/upload; POST /logs/ingest; GET /logs/search; GET /logs/batches
Detection and alerts	GET /api/v1/detections; GET /detections/alerts; GET /detections/rules; POST /detections/run
ML	POST /api/v1/ml/train (admin); POST /ml/predict; GET /ml/models; GET /ml/models/latest; POST /ml/evaluate (admin)
Incidents	GET /api/v1/incidents; GET /incidents/{ref}; PATCH status; POST notes; PATCH assign
Analytics	GET /api/v1/analytics/summary, /top-ips, /severity-distribution, /threat-timeline, /attack-types, /recent-events
Rule administration	GET /api/v1/rules; PATCH /rules/{rule_name} (admin)

6.2 Data-store map

Question	Project-specific answer
MongoDB	users, incidents, detection_rules, system_config, ingestion_batches, detections, alerts, counters
Elasticsearch	security-logs-YYYY.MM.DD daily normalized evidence indices queried via security-logs-*
Filesystem artifacts	ml/models/isolation_forest_vN.joblib plus ml/models/registry_index.json; mounted into backend by Compose

6.3 Pre-viva verification checklist

- Run docker compose up -d --build and confirm backend, frontend, Mongo, and Elasticsearch are healthy.
- Call /health and /health/ready; explain liveness versus downstream readiness.
- Seed the admin and demo data only if your environment requires it, then change default passwords for shared use.
- Prepare each sample source: linux_auth.log, windows_sample.json, firewall_sample.log, web_access.log.
- Know one implemented limitation and one future improvement; never claim external alerts, full EVTX binary support, or globally calibrated ML probability when the code does not implement them.

End of developer and viva guide.